Trabajo tutorizado

YOLO

Tecnologías Avanzadas

Profesor José Antonio Lagares Rodríguez
Máster Big Data & Business Analytics (MBDA)





Alumno: ÁLVARO RAMOS BERNABÉ

ENUNCIADO DE LA ACTIVIDAD

Guion oficial: «MBDA - Trabajo tutorizado - YOLO.pdf» (Campus Virtual UPO).

Cargue y ejecute el notebook “Ej1” adjunto en el Aula Virtual dentro de Google Colab. Fíjese cómo se anota dentro de las imágenes.

Una vez analizado, responda a las siguientes preguntas:

1) ¿Qué contiene el archivo “data.yaml” y por qué es importante?

2) ¿Qué ocurre después del entrenamiento?

3) ¿Dónde se guardan los resultados?

4) ¿Qué significan las métricas que aparecen al final (ejemplo: mAP, precisión, recall)?

5) ¿Qué ocurre si se baja el entrenamiento a 1 época? ¿es suficiente? ¿Y con 5 épocas?

Ejercicio 2 – 5 Puntos – 120 minutos Crea un cuaderno de código en Google Colab para entrenar y probar tu propio modelo de Yolo. Puedes usar un dataset de internet o crear el tuyo propio.

Ten en cuenta los siguientes puntos:

1) Recuerda que si haces una segmentación o usas cajas delimitadoras deberás trabajar con alguna herramienta de anotación. Deja la carpeta “train” y “val” lista con las fotos y las anotaciones si son requeridas.

2) Empieza con pocas épocas para realizar un entrenamiento base. Posteriormente, suba el número de épocas. Analice la pérdida de error en todos los casos.

3) Pruebe el modelo entrenado. Si es posible, use un video para ver el desempeño en tiempo real.

Fecha y Modo de entrega Los alumnos entregarán un documento PDF a través de una actividad programada en el Aula Virtual empleando la plantilla suministrada.

RESOLUCIÓN

Los dos cuadernos se han ejecutado de principio a fin con Ultralytics 8.4 y PyTorch 2.14, y se entregan junto a este documento con todas sus salidas. Se ha usado Jupyter con el mismo código que en Google Colab, sustituyendo las celdas propias de Colab (montaje de Drive) por rutas locales. Al no disponer de GPU, el entrenamiento se hace en CPU, lo que alarga los tiempos pero no altera los resultados.

Ejercicio 1. Cuaderno «YOLO - Ej1»: detección de extremo a extremo

El cuaderno recorre el ciclo completo: dataset en formato YOLO → normalización del `data.yaml` → entrenamiento de YOLO11n → métricas → mAP con `supervision` → visualización. El dataset del guion (`circuit-voltages`, Roboflow) exige iniciar sesión con una API key de Roboflow. Por eso se ha usado **medical-pills**, un dataset público de Ultralytics con la misma estructura: 1 clase (`pill`), 92 imágenes de entrenamiento y 23 de validación con 399 pastillas anotadas.

```
In [2]: dataset_location = f"{HOME}/medical-pills"
        !ls {dataset_location} {dataset_location}/images
        !ls {dataset_location}/images/train | head -3; ls {dataset_location}/labels/train | wc -l
        !head -3 {dataset_location}/labels/train/Frame_0.txt
```

Figura 1. Estructura del dataset y ejemplo de fichero de etiquetas YOLO (clase, x_centro, y_centro, ancho, alto normalizados).

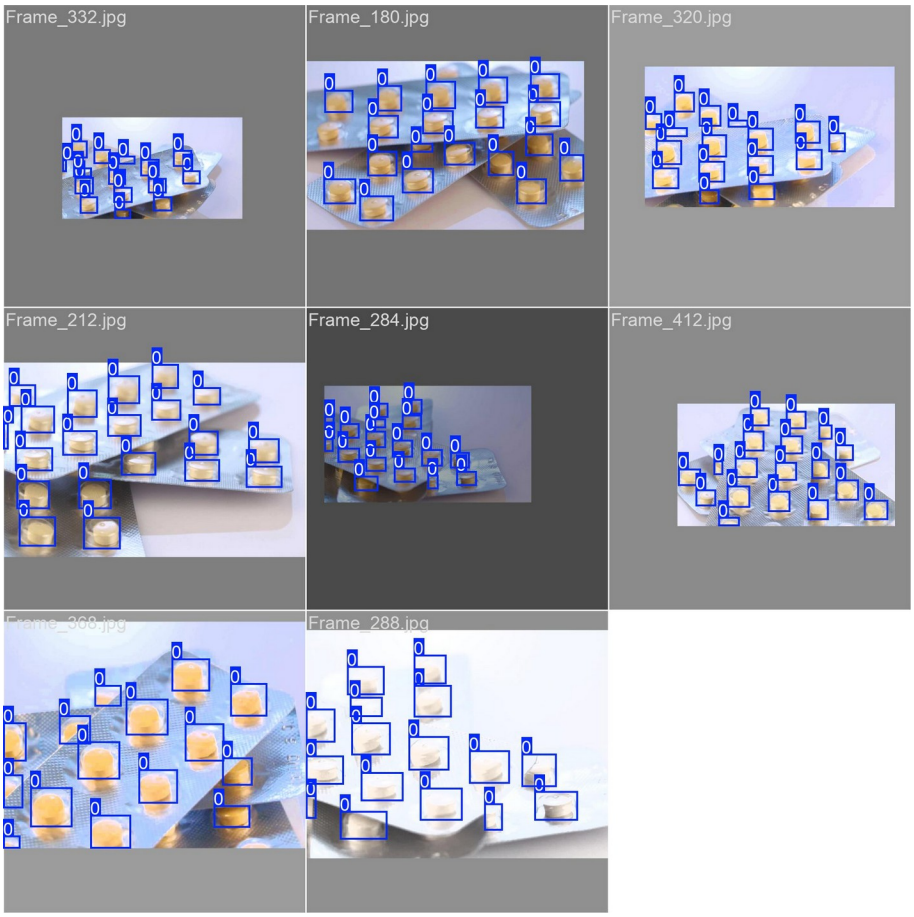


Figura 2. Lote de entrenamiento con las anotaciones dibujadas (mosaico y aumentos que aplica YOLO). Así se «anota» dentro de las imágenes: una caja por objeto.

```
In [3]: import yaml, shutil
shutil.copy(f'{dataset_location}/medical-pills.yaml', f'{dataset_location}/data.yaml')
yaml_path = f'{dataset_location}/data.yaml'
with open(yaml_path) as f:
    data = yaml.safe_load(f)
data['path'] = dataset_location
data['train'] = 'images/train'
data['val'] = 'images/val'
data.pop('download', None)
with open(yaml_path, 'w') as f:
    yaml.dump(data, f, sort_keys=False)
!cat {yaml_path}
```

Figura 3. Normalización del `data.yaml`: rutas relativas de train/val, número y nombre de clases.

```
In [4]: from ultralytics import YOLO
model = YOLO('yolo11n.pt')
results = model.train(data=yaml_path, epochs=10, imgsz=640, batch=8, workers=2, device='cpu', name='train_e10', exist_ok=True, plots=True
```

```
[... salida intermedia omitida ...]
      1/10         0G      1.36      2.868      1.152         66        640: 100% ━━━━━━━━━━━━ 12/12 3.4s/it 40.6s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 1.0s/it 2.1s
                   all         23        399     0.0574      0.992      0.709      0.357

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       2/10         0G      1.163      1.805     0.9862         63        640: 100% ━━━━━━━━━━━━ 12/12 3.0s/it 36.2s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 1.0it/s 1.9s
                   all         23        399     0.0572       0.99       0.83      0.412

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       3/10         0G      1.227      1.276      1.033         66        640: 100% ━━━━━━━━━━━━ 12/12 3.0s/it 35.6s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 1.0s/it 2.0s
                   all         23        399     0.0575      0.995      0.753      0.398

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       4/10         0G      1.139     0.9995      1.029         63        640: 100% ━━━━━━━━━━━━ 12/12 3.1s/it 37.1s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 1.0s/it 2.1s
                   all         23        399       0.11      0.977        0.9      0.608

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       5/10         0G      1.074     0.8786       1.02         66        640: 100% ━━━━━━━━━━━━ 12/12 3.0s/it 36.6s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 1.2s/it 2.4s
                   all         23        399      0.997      0.856      0.979        0.7

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       6/10         0G      1.007     0.8123      1.006         66        640: 100% ━━━━━━━━━━━━ 12/12 2.9s/it 35.4s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 1.1s/it 2.1s
                   all         23        399      0.997      0.919      0.982       0.69

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       7/10         0G     0.9588      0.767     0.9786         69        640: 100% ━━━━━━━━━━━━ 12/12 3.0s/it 36.4s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 1.1s/it 2.2s
                   all         23        399      0.987      0.967      0.988      0.725

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       8/10         0G     0.9355     0.7417     0.9686         66        640: 100% ━━━━━━━━━━━━ 12/12 3.1s/it 36.7s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 1.1s/it 2.2s
                   all         23        399      0.987      0.977       0.99      0.739

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       9/10         0G     0.9046     0.7268     0.9744         72        640: 100% ━━━━━━━━━━━━ 12/12 3.1s/it 36.7s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 1.0s/it 2.1s
                   all         23        399      0.978      0.981      0.992      0.744

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      10/10         0G      0.911     0.6896     0.9618         71        640: 100% ━━━━━━━━━━━━ 12/12 3.1s/it 37.8s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 1.5s/it 3.0s
                   all         23        399      0.976      0.985      0.992      0.752
10 epochs completed in 0.109 hours.
Optimizer stripped from /home/aramber/ta04/ej1/runs/detect/train_e10/weights/last.pt, 5.4MB
Optimizer stripped from /home/aramber/ta04/ej1/runs/detect/train_e10/weights/best.pt, 5.4MB
Validating /home/aramber/ta04/ej1/runs/detect/train_e10/weights/best.pt...
Ultralytics 8.4.156 🚀 Python-3.11.15 torch-2.14.0+cu130 CPU (Intel Xeon Processor @ 2.10GHz)
YOLO11n summary (fused): 100 layers, 2,582,347 parameters, 0 gradients, 6.4 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 1.4s/it 2.8s
                   all         23        399      0.976      0.985      0.992      0.752
Speed: 1.8ms preprocess, 87.0ms inference, 0.0ms loss, 13.3ms postprocess per image
Results saved to /home/aramber/ta04/ej1/runs/detect/train_e10
```

Figura 4. Entrenamiento de 10 épocas: pérdidas (box, cls, dfl) y métricas de validación en cada época. Resultado final: P = 0,976, R = 0,985, mAP50 = 0,992 y mAP50-95 = 0,752.

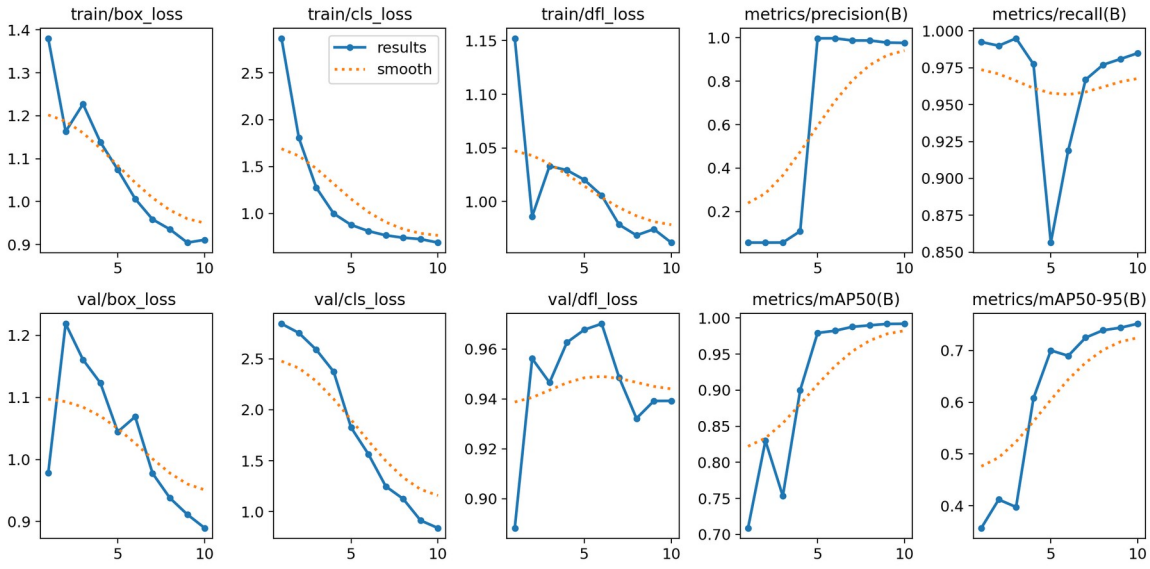


Figura 5. Curvas de `results.png`: las pérdidas de entrenamiento y validación bajan de forma sostenida y el mAP sube hasta estabilizarse.

```
In [7]: predictions, targets = [], []
        for _, image, target in ds:
            r = model(image, verbose=False)[0]
            predictions.append(sv.Detections.from_ultralytics(r)); targets.append(target)
        mapr = MeanAveragePrecision().update(predictions, targets).compute()
        print(f'mAP@50:95: {mapr.map50_95:.3f}')
        print(f'mAP@50:    {mapr.map50:.3f}')
        print(f'mAP@75:    {mapr.map75:.3f}')

mAP@50:95: 0.728
mAP@50:    0.948
mAP@75:    0.908
```

Figura 6. mAP calculado con `supervision` sobre la validación: mAP@50 = 0,948, mAP@75 = 0,908, mAP@50:95 = 0,728.



Figura 7. Predicción sobre una imagen de validación: 16 pastillas detectadas de 16 reales.

Respuestas

1) ¿Qué contiene el archivo `data.yaml` y por qué es importante? Es el descriptor del dataset. Indica la ruta raíz (`path`), las carpetas de imágenes de entrenamiento, validación y, si existe, test (`train`, `val`, `test`), y el diccionario de clases (`names`, aquí `0: pill`), del que se deduce su número (`nc`). YOLO lo usa para localizar las imágenes y, por convención, sus etiquetas en la carpeta `labels/` paralela. También lo usa para **redimensionar la cabeza de detección** al número de clases: en el log aparece *«Overriding model.yaml nc=80 with nc=1»*, porque el modelo preentrenado en COCO tenía 80 clases. Si las rutas o las clases son incorrectas, el entrenamiento falla o aprende etiquetas equivocadas. Por eso el cuaderno lo «normaliza» a rutas relativas.

2) ¿Qué ocurre después del entrenamiento? Ultralytics guarda los pesos `last.pt` (última época) y `best.pt` (la de mejor *fitness*, que combina 0,1·mAP50 y 0,9·mAP50-95), les quita el estado del optimizador para reducir su tamaño (5,4 MB) y **valida automáticamente `best.pt`** sobre el conjunto de validación. Imprime las métricas finales y la velocidad de inferencia (≈ 87 ms/imagen en CPU) y genera gráficas: `results.png`, matriz de confusión, curvas P, R, F1 y PR, y mosaicos de lotes con predicciones. Después, el cuaderno carga `best.pt` para calcular el mAP con `supervision` y dibujar predicciones.

3) ¿Dónde se guardan los resultados? En `runs/detect/<nombre>/`, en este caso `runs/detect/train_e10/`. Contiene `weights/best.pt` y `weights/last.pt`, `results.csv` (métricas por época), `results.png`, `confusion_matrix*.png`, las curvas `Box*_curve.png`, `labels.jpg` y los mosaicos `train_batch*.jpg` y

`val_batch*_pred.jpg`, además de `args.yaml` con la configuración usada. Cada nueva ejecución crea `train2`, `train3`… salvo que se fije `name` y `exist_ok=True`.

4) ¿Qué significan las métricas?

- **Precisión (P):** de las cajas predichas, qué fracción es correcta (TP / (TP + FP)). Mide los falsos positivos.
- **Recall (R):** de los objetos reales, qué fracción se detecta (TP / (TP + FN)). Mide los objetos perdidos.
- **mAP50:** media, entre clases, del área bajo la curva precisión-recall, considerando acierto una caja con IoU ≥ 0,5 respecto a la real.
- **mAP50-95:** el mismo cálculo promediado para umbrales de IoU de 0,50 a 0,95 en pasos de 0,05. Es mucho más exigente con la **precisión de la localización**; por eso aquí vale 0,752 frente a 0,992 del mAP50.
- **Pérdidas:** `box_loss` (error de las coordenadas de la caja, CIoU), `cls_loss` (error de clasificación) y `dfl_loss` (*Distribution Focal Loss*, que afina los bordes).

5) ¿1 época? ¿5 épocas?

```
In [9]: res = {}
for ep in [1, 5]:
    m = YOLO('yolo11n.pt')
    m.train(data=yaml_path, epochs=ep, imgsz=640, batch=8, workers=2, device='cpu', name=f'train_e{ep}', exist_ok=True, plots=False, verb
    met = YOLO(f'{HOME}/runs/detect/train_e{ep}/weights/best.pt').val(data=yaml_path, device='cpu', verbose=False, plots=False)
    res[ep] = (met.box.mp, met.box.mr, met.box.map50, met.box.map)
met = YOLO(best).val(data=yaml_path, device='cpu', verbose=False, plots=False)
res[10] = (met.box.mp, met.box.mr, met.box.map50, met.box.map)
import pandas as pd
print(pd.DataFrame(res, index=['precision','recall','mAP50','mAP50-95']).T.round(3))
Ultralytics 8.4.156 🚀 Python-3.11.15 torch-2.14.0+cu130 CPU (Intel Xeon Processor @ 2.10GHz)
[... salida intermedia omitida ...]
Ultralytics 8.4.156 🚀 Python-3.11.15 torch-2.14.0+cu130 CPU (Intel Xeon Processor @ 2.10GHz)
YOLO11n summary (fused): 100 layers, 2,582,347 parameters, 0 gradients, 6.4 GFLOPs
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 3041.7±558.0 MB/s, size: 75.9 KB)
val: Scanning /home/aramber/ta04/ej1/medical-pills/labels/val.cache... 23 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 23/23 9.6Mi
t/s 0.0s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 1.3it/s 1.5s
                   all         23        399      0.976      0.985      0.992      0.752
Speed: 0.9ms preprocess, 48.1ms inference, 0.0ms loss, 10.1ms postprocess per image
    precision  recall  mAP50  mAP50-95
1       0.000   0.000  0.000     0.000
5       0.959   0.877  0.918     0.654
10      0.976   0.985  0.992     0.752
```

Figura 8. Reentrenamiento desde el modelo preentrenado con 1 y 5 épocas y comparación con 10.

Épocas	Precisión	Recall	mAP50	mAP50-95
1	0,000	0,000	0,000	0,000
5	0,959	0,877	0,918	0,654
10	0,976	0,985	0,992	0,752

Con 1 época no es suficiente. La cabeza de detección se reinicia para la nueva clase y, tras una sola pasada con la tasa de aprendizaje aún en *warm-up*, el modelo no produce detecciones con confianza suficiente: todas las métricas son 0. **Con 5 épocas** ya es útil (mAP50 = 0,918), pero pierde un 12 % de pastillas (R = 0,877) y localiza peor (mAP50-95 = 0,654). **Con 10 épocas** casi satura el mAP50 y mejora la localización. Más épocas mejorarían todavía el mAP50-95, vigilando el sobreajuste con la pérdida de validación, que aquí sigue bajando.

Ejercicio 2. Modelo YOLO propio: clasificación del tiempo atmosférico

Se entrena un **clasificador YOLO11n-cls** que reconoce cuatro situaciones meteorológicas en fotografías: **amanecer, lluvioso, nublado y soleado**. Se usa el dataset `dataset_tiempo` del

Aula Virtual: 948 imágenes de entrenamiento y 176 de validación. Al tratarse de **clasificación** no se necesitan cajas ni polígonos: la «anotación» consiste en dejar cada imagen en la carpeta de su clase, dentro de `train/` y `val/`, que es el formato que espera YOLO.

```
In [1]: import os, glob, collections, torch, ultralytics
        HOME = os.getcwd(); DATA = f"{HOME}/dataset_tiempo"
        print("ultralytics", ultralytics.__version__, "| dispositivo:", "cuda" if torch.cuda.is_available() else "cpu")
        for split in ["train", "val"]:
            cont = {c: len(os.listdir(f"{DATA}/{split}/{c}")) for c in sorted(os.listdir(f"{DATA}/{split}"))}
            print(split, cont, "total =", sum(cont.values()))
```

```
ultralytics 8.4.156 | dispositivo: cpu
train {'amanecer': 300, 'lluvioso': 199, 'nublado': 249, 'soleado': 200} total = 948
val {'amanecer': 56, 'lluvioso': 16, 'nublado': 51, 'soleado': 53} total = 176
```

Figura 9. Recuento por clase y partición.

```
In [2]: !find dataset_tiempo -maxdepth 2 -type d | sort
        !ls dataset_tiempo/train/soleado | head -3
```

Figura 10. Estructura de carpetas `train/<clase>` y `val/<clase>` lista para el entrenamiento.

Entrenamiento base (pocas épocas) y entrenamiento ampliado

```
In [3]: from ultralytics import YOLO
        base = YOLO('yolo11n-cls.pt')
        base.train(data=DATA, epochs=3, imgsz=224, batch=32, workers=2, device='cpu', name='base_e3', exist_ok=True, verbose=False, plots=True)
```

```
[... salida intermedia omitida ...]
                   all      0.972          1
3 epochs completed in 0.029 hours.
Optimizer stripped from /home/aramber/ta04/ej2/runs/classify/base_e3/weights/last.pt, 3.2MB
Optimizer stripped from /home/aramber/ta04/ej2/runs/classify/base_e3/weights/best.pt, 3.2MB
Validating /home/aramber/ta04/ej2/runs/classify/base_e3/weights/best.pt...
Ultralytics 8.4.156 🚀 Python-3.11.15 torch-2.14.0+cu130 CPU (Intel Xeon Processor @ 2.10GHz)
YOLO11n-cls summary (fused): 47 layers, 1,531,148 parameters, 0 gradients, 3.2 GFLOPs
train: /home/aramber/ta04/ej2/dataset_tiempo/train... found 948 images in 4 classes ✅
val: /home/aramber/ta04/ej2/dataset_tiempo/val... found 176 images in 4 classes ✅
test: None...
               classes   top1_acc   top5_acc: 100% ━━━━━━━━━━━━ 3/3 1.9it/s 1.6s
                   all      0.972          1
Speed: 0.0ms preprocess, 5.6ms inference, 0.0ms loss, 0.0ms postprocess per image
Results saved to /home/aramber/ta04/ej2/runs/classify/base_e3
```

```
Out[3]: ultralytics.utils.metrics.ClassifyMetrics object with attributes:

        confusion_matrix: <ultralytics.utils.metrics.ConfusionMatrix object at 0x7ff49f87d210>
        curves: []
        curves_results: []
        fitness: 0.9857954680919647
        keys: ['metrics/accuracy_top1', 'metrics/accuracy_top5']
        results_dict: {'metrics/accuracy_top1': 0.9715909361839294, 'metrics/accuracy_top5': 1.0, 'fitness': 0.9857954680919647}
        save_dir: PosixPath('/home/aramber/ta04/ej2/runs/classify/base_e3')
        speed: {'preprocess': 0.0005126079465547016, 'inference': 5.6486171761207515, 'loss': 3.4863629397560466e-05, 'postprocess': 6.462500385
        334185e-05}
        top1: 0.9715909361839294
        top5: 1.0
```

Figura 11. Entrenamiento base de 3 épocas (imgsz = 224): top-1 = 0,972 en validación.

```
In [4]: modelo = YOLO('yolo11n-cls.pt')
        modelo.train(data=DATA, epochs=15, imgsz=224, batch=32, workers=2, device='cpu', name='final_e15', exist_ok=True, verbose=False, plots=Tr
```

```
Ultralytics 8.4.156 🚀 Python-3.11.15 torch-2.14.0+cu130 CPU (Intel Xeon Processor @ 2.10GHz)
[... salida intermedia omitida ...]
      Epoch    GPU_mem       loss  Instances       Size
      14/15         0G    0.05839         18        224: 100% ━━━━━━━━━━━━ 30/30 1.1s/it 32.2s
                classes   top1_acc   top5_acc: 100% ━━━━━━━━━━━━ 3/3 1.4it/s 2.2s
                    all      0.972          1
      Epoch    GPU_mem       loss  Instances       Size
      15/15         0G    0.04953         18        224: 100% ━━━━━━━━━━━━ 30/30 1.1s/it 32.4s
                classes   top1_acc   top5_acc: 100% ━━━━━━━━━━━━ 3/3 1.4it/s 2.2s
                    all      0.966          1
15 epochs completed in 0.144 hours.
Optimizer stripped from /home/aramber/ta04/ej2/runs/classify/final_e15/weights/last.pt, 3.2MB
Optimizer stripped from /home/aramber/ta04/ej2/runs/classify/final_e15/weights/best.pt, 3.2MB
Validating /home/aramber/ta04/ej2/runs/classify/final_e15/weights/best.pt...
Ultralytics 8.4.156 🚀 Python-3.11.15 torch-2.14.0+cu130 CPU (Intel Xeon Processor @ 2.10GHz)
YOLO11n-cls summary (fused): 47 layers, 1,531,148 parameters, 0 gradients, 3.2 GFLOPs
train: /home/aramber/ta04/ej2/dataset_tiempo/train... found 948 images in 4 classes ✅
val: /home/aramber/ta04/ej2/dataset_tiempo/val... found 176 images in 4 classes ✅
test: None...
                classes   top1_acc   top5_acc: 100% ━━━━━━━━━━━━ 3/3 1.9it/s 1.6s
                    all      0.983          1
Speed: 0.0ms preprocess, 5.7ms inference, 0.0ms loss, 0.0ms postprocess per image
Results saved to /home/aramber/ta04/ej2/runs/classify/final_e15
```

```
Out[4]: ultralytics.utils.metrics.ClassifyMetrics object with attributes:

confusion_matrix: <ultralytics.utils.metrics.ConfusionMatrix object at 0x7ff48383ec10>
curves: []
curves_results: []
fitness: 0.9914772808551788
keys: ['metrics/accuracy_top1', 'metrics/accuracy_top5']
results_dict: {'metrics/accuracy_top1': 0.9829545617103577, 'metrics/accuracy_top5': 1.0, 'fitness': 0.9914772808551788}
save_dir: PosixPath('/home/aramber/ta04/ej2/runs/classify/final_e15')
speed: {'preprocess': 0.0005121534059071564, 'inference': 5.7397659886336525, 'loss': 2.4011373253332834e-05, 'postprocess': 6.921590573
421615e-05}
top1: 0.9829545617103577
top5: 1.0
```

Figura 12. Entrenamiento ampliado a 15 épocas: top-1 = 0,983 con los mejores pesos.

```
In [5]: import pandas as pd, matplotlib.pyplot as plt
        r3 = pd.read_csv('runs/classify/base_e3/results.csv'); r15 = pd.read_csv('runs/classify/final_e15/results.csv')
        print(r15[['epoch','train/loss','val/loss','metrics/accuracy_top1']].round(4).to_string(index=False))
        fig, ax = plt.subplots(1, 2, figsize=(12, 4))
        for r, n in [(r3, '3 épocas'), (r15, '15 épocas')]:
            ax[0].plot(r['epoch'], r['train/loss'], marker='o', label=f'train {n}'); ax[0].plot(r['epoch'], r['val/loss'], ls='--', marker='x', l
            ax[1].plot(r['epoch'], r['metrics/accuracy_top1'], marker='o', label=n)
        ax[0].set_title('Pérdida (cross-entropy)'); ax[0].set_xlabel('época'); ax[0].legend()
        ax[1].set_title('Accuracy top-1 en validación'); ax[1].set_xlabel('época'); ax[1].legend()
        plt.tight_layout(); plt.savefig('perdidas.png', dpi=110); plt.show()
```

epoch	train/loss	val/loss	metrics/accuracy_top1
1	0.8315	0.1324	0.9602
2	0.1981	0.0847	0.9659
3	0.1727	0.2254	0.9318
4	0.1647	0.1604	0.9602
5	0.1821	0.1488	0.9773
6	0.1532	0.1266	0.9773
7	0.1292	0.2541	0.9489
8	0.1083	0.0776	0.9830
9	0.1162	0.0827	0.9773
10	0.1109	0.0759	0.9716
11	0.0824	0.0635	0.9773
12	0.0530	0.0732	0.9773
13	0.0599	0.0657	0.9773
14	0.0584	0.0793	0.9716
15	0.0495	0.0968	0.9659

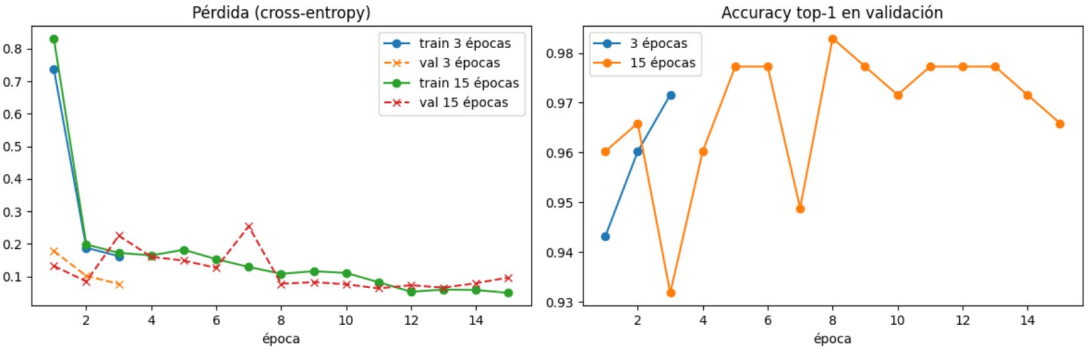


Figura 13. Evolución de la pérdida y de la accuracy por época (tabla de `results.csv`).

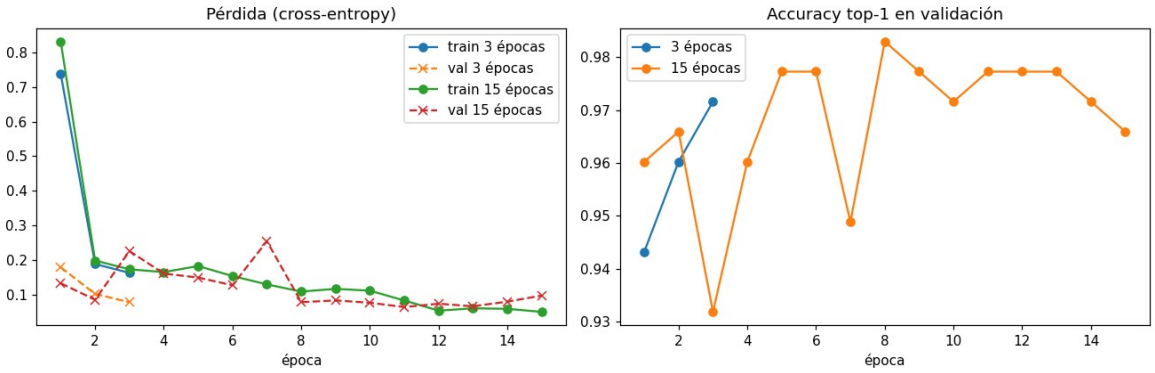


Figura 14. Análisis de la pérdida: entrenamiento y validación para 3 y 15 épocas, y accuracy top-1.

Análisis de la pérdida. Partir de pesos preentrenados en ImageNet hace que el modelo aprenda muy rápido: con solo 3 épocas alcanza un 97,2 % de acierto. Con 15 épocas, la pérdida de **entrenamiento** baja de forma continua (de 0,17 a 0,05). La de **validación** baja hasta ~0,06 en la época 11 y después oscila ligeramente al alza (0,07-0,10) mientras la de entrenamiento sigue bajando: es el inicio del **sobreajuste**. Por eso Ultralytics conserva como `best.pt` la época con mejor métrica de validación, no la última. Para este dataset, 8-12 épocas son el punto óptimo.

```
In [6]: best = YOLO('runs/classify/final_e15/weights/best.pt')
        m = best.val(data=DATA, imgsz=224, device='cpu', verbose=False, plots=True)
        print(f'top-1 = {m.top1:.3f} | top-5 = {m.top5:.3f}')
        b3 = YOLO('runs/classify/base_e3/weights/best.pt').val(data=DATA, imgsz=224, device='cpu', verbose=False, plots=False)
        print(f'Modelo base (3 épocas) top-1 = {b3.top1:.3f}')

Ultralytics 8.4.156 🚀 Python-3.11.15 torch-2.14.0+cu130 CPU (Intel Xeon Processor @ 2.10GHz)
YOLO11n-cls summary (fused): 47 layers, 1,531,148 parameters, 0 gradients, 3.2 GFLOPs
train: /home/aramber/ta04/ej2/dataset_tiempo/train... found 948 images in 4 classes ✅
val: /home/aramber/ta04/ej2/dataset_tiempo/val... found 176 images in 4 classes ✅
test: None...
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 520.7±263.3 MB/s, size: 5.2 KB)
val: Scanning /home/aramber/ta04/ej2/dataset_tiempo/val... 176 images, 0 corrupt: 100% ━━━━━━━━━━━━ 176/176 67.1Mit/s 0.0s
               classes   top1_acc   top5_acc: 100% ━━━━━━━━━━━━ 11/11 7.7it/s 1.4s
                   all      0.983          1
Speed: 0.0ms preprocess, 4.7ms inference, 0.0ms loss, 0.0ms postprocess per image
Results saved to /home/aramber/ta04/ej2/runs/classify/val
top-1 = 0.983 | top-5 = 1.000
Ultralytics 8.4.156 🚀 Python-3.11.15 torch-2.14.0+cu130 CPU (Intel Xeon Processor @ 2.10GHz)
YOLO11n-cls summary (fused): 47 layers, 1,531,148 parameters, 0 gradients, 3.2 GFLOPs
train: /home/aramber/ta04/ej2/dataset_tiempo/train... found 948 images in 4 classes ✅
val: /home/aramber/ta04/ej2/dataset_tiempo/val... found 176 images in 4 classes ✅
test: None...
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 548.1±309.1 MB/s, size: 5.2 KB)
val: Scanning /home/aramber/ta04/ej2/dataset_tiempo/val... 176 images, 0 corrupt: 100% ━━━━━━━━━━━━ 176/176 73.8Mit/s 0.0s
               classes   top1_acc   top5_acc: 100% ━━━━━━━━━━━━ 11/11 9.0it/s 1.2s
                   all      0.972          1
Speed: 0.0ms preprocess, 4.5ms inference, 0.0ms loss, 0.0ms postprocess per image
Modelo base (3 épocas) top-1 = 0.972
```

Figura 15. Validación final de `best.pt` (top-1 = 0,983; top-5 = 1,0) frente al modelo base de 3 épocas (0,972).

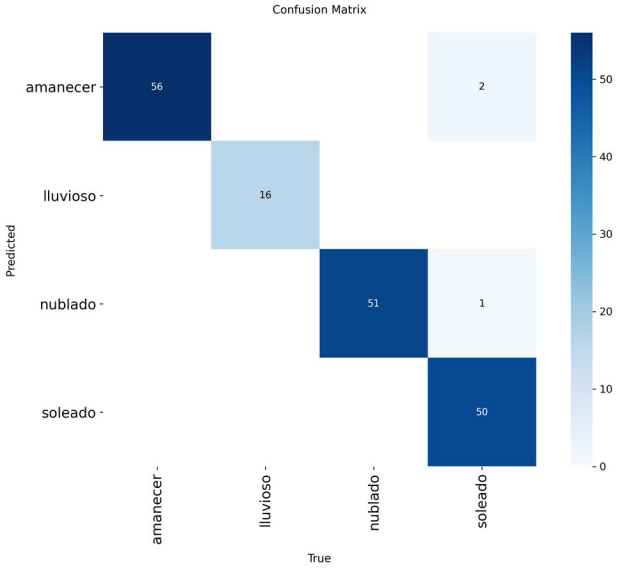


Figura 16. Matriz de confusión en validación: solo 3 errores de 176, todos en imágenes reales «soleado» (2 predichas como «amanecer» y 1 como «nublado»).

Prueba del modelo en imágenes y en vídeo

```
In [7]: import random, cv2
        random.seed(7)
        imgs = [random.choice(glob.glob(f'{DATA}/val/{c}/*')) for c in ['amanecer','lluvioso','nublado','soleado']]
        for p in imgs:
            r = best(p, verbose=False)[0]
            print(f"{p.split('/')[-2]:9s} -> predicho: {r.names[r.probs.top1]:9s} ({r.probs.top1conf:.2f})")

amanecer  -> predicho: amanecer  (1.00)
lluvioso  -> predicho: lluvioso  (1.00)
nublado   -> predicho: nublado   (1.00)
soleado   -> predicho: soleado   (1.00)
```

Figura 17. Predicción sobre una imagen aleatoria de validación de cada clase.

Para comprobar el funcionamiento en tiempo real se compone un vídeo de 24 s (12 fotografías de validación, 2 s cada una, a 10 fps). El modelo lo procesa fotograma a fotograma, escribiendo sobre cada uno la clase predicha y su confianza, y genera `salida_predicciones.mp4`, que se entrega junto al cuaderno. La inferencia tarda unos 5 ms por fotograma en CPU, muy por debajo de los 100 ms de un vídeo a 10 fps.

```
In [8]: random.seed(1)
        sel = [f for c in ['soleado','nublado','lluvioso','amanecer'] for f in random.sample(glob.glob(f'{DATA}/val/{c}/*'), 3)]
        W, H, fps = 640, 400, 10
        vw = cv2.VideoWriter('entrada.mp4', cv2.VideoWriter_fourcc(*'mp4v'), fps, (W, H))
        for f in sel:
            im = cv2.resize(cv2.imread(f), (W, H))
            for _ in range(2 * fps): vw.write(im)
        vw.release()
        out = cv2.VideoWriter('salida_predicciones.mp4', cv2.VideoWriter_fourcc(*'mp4v'), fps, (W, H))
        cap = cv2.VideoCapture('entrada.mp4'); n = 0; muestras = []
        while True:
            ok, fr = cap.read()
            if not ok: break
            r = best(fr, verbose=False)[0]; lab = f"{r.names[r.probs.top1]} {r.probs.top1conf:.2f}"
            cv2.rectangle(fr, (0, 0), (W, 42), (0, 0, 0), -1); cv2.putText(fr, lab, (12, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
            out.write(fr); n += 1
            if n % (2 * fps) == 10: muestras.append(fr.copy())
        out.release()
        cv2.imwrite('fotogramas_video.jpg', cv2.vconcat([cv2.hconcat(muestras[i:i+4]) for i in range(0, 12, 4)]))
        print('fotogramas procesados:', n, '| vídeo generado: salida_predicciones.mp4')

fotogramas procesados: 240 | vídeo generado: salida_predicciones.mp4
```

Figura 18. Código de la prueba en vídeo: 240 fotogramas procesados.



Figura 19. Fotogramas del vídeo de salida con la predicción superpuesta.

En el vídeo acierta 11 de las 12 escenas. El único fallo es una imagen de la carpeta «soleado» (una flor amarilla en primer plano con luz cálida) que el modelo clasifica como «amanecer». Es el mismo patrón de error que muestra la matriz de confusión y resulta razonable: el modelo aprende sobre todo de la **tonalidad y la luz global** y esa foto tiene la paleta cálida típica de los amaneceres. Mejoraría con más ejemplos soleados de primer plano o con aumentos de color (`hsv_h`, `hsv_s`) más agresivos.

Referencias

Ultralytics YOLO Docs – Train, Val, Predict; Detect y Classify: https://docs.ultralytics.com

Ultralytics – Medical Pills Dataset: https://docs.ultralytics.com/datasets/detect/medical-pills/

Roboflow `supervision` – MeanAveragePrecision: https://supervision.roboflow.com

Padilla, R. et al. (2021). A comparative analysis of object detection metrics. *Electronics*, 10(3), 279.